# ALGORITHMIC COMPLEXITY

*Comprehensive Solutions Guide & Analysis*

## Section 1: Iterative Code Analysis (Q1 - Q13)

### Question 1

Determine the time complexity:

```
for(int i=0;i<n;i++)
{
    cout<<i;
}
```

**Time Complexity: O(n)**

**Explanation:** The loop control variable ***i*** starts at 0 and increments by 1 on each iteration (***i***++). The loop terminates when ***i*** reaches ***n***. Thus, the statement inside the loop executes exactly ***n*** times. Each execution takes constant ***O(1)*** time, resulting in a linear time complexity.

### Question 2

Determine the time complexity:

```
for(int i=0;i<n;i++)
{
    for(int j=0;j<n;j++)
    {
        cout<<"*";
    }
}
```

**Time Complexity: O(n²)**

**Explanation:** This contains two nested loops. The outer loop runs ***n*** times (from 0 to ***n-1***). For every single iteration of the outer loop, the inner loop runs independently ***n*** times (from 0 to ***n-1***). The total number of executions is ***n*** × ***n*** = ***n²***, yielding a quadratic time complexity.

### Question 3

Determine the time complexity:

```
for(int i=0;i<n;i++)
{
    for(int j=0;j<i;j++)
    {
        cout<<"*";
    }
}
```

**Time Complexity: $O(n^2)$**

**Explanation:** The outer loop runs $n$ times. The inner loop depends on the current value of $i$, running $i$ times. The total operations can be written as the sum series:

$$0 + 1 + 2 + 3 + ... + (n-1) = [n(n-1)] / 2 = 0.5n^2 - 0.5n$$

Dropping the lower-order terms and constant coefficients gives $O(n^2)$.

### Question 4

Determine the time complexity:

```
for(int i=1;i<n;i*=2)
{
    cout<<i;
}
```

**Time Complexity: $O(\log_2 n)$**

**Explanation:** The loop variable $i$ starts at 1 and is multiplied by 2 in each iteration ($i = 1, 2, 4, 8, ..., 2^k$). The loop stops when $2^k \geq n$. Taking the logarithm on both sides: $k = \log_2 n$. Therefore, the loop runs logarithmic times.

### Question 5

Determine the time complexity:

```
for(int i=n;i>1;i/=2)
{
    cout<<i;
}
```

**Time Complexity: O($\log_2 n$)**

**Explanation:** This is the inverse of Q4. The variable $i$ starts at $n$ and is divided by 2 in each iteration until it drops to 1. The sequence of values is $n, n/2, n/4, ..., n/2^k$. Setting $n/2^k = 1$ gives $k = \log_2 n$ steps.

### Question 6

Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=log(n);j++)
    {
        cout<<"*";
    }
}
```

**Time Complexity: O(n log n)**

**Explanation:** The outer loop is linear and executes exactly $n$ times. The inner loop executes $log(n)$ times for every outer loop iteration. Since the inner loop limit doesn't depend on $i$, the total operations are simply the product of both iteration bounds: $n \times log(n) = O(n \log n)$.

### Question 7

Determine the time complexity:

```
while(n>0)
{
    n--;
}
```

**Time Complexity: O(n)**

**Explanation:** The variable $n$ decrements by 1 each time. The loop runs continuously from the initial value of $n$ down to 0. This results in exactly $n$ iterations, which represents linear $O(n)$ execution time.

### Question 8

Determine the time complexity:

```
while(n>1)
{
    n/=2;
}
```

**Time Complexity: O(log n)**

**Explanation:** The value of $n$ is halved during each loop cycle. As shown in Q5, continuously dividing a value by a constant factor until reaching a constant threshold takes logarithmic steps, or $O(\log_2 n)$.

### Question 9

Determine the time complexity:

```
for(int i=1;i<=n;i++)
{}
for(int j=1;j<=n;j++)
{}
```

**Time Complexity: O(n)**

**Explanation:** These two loops are sequential, not nested. The first loop runs $n$ times, and the second loop runs $n$ times. The combined total time complexity is $O(n) + O(n) = O(2n) = O(n)$, as constant multipliers are omitted in Big-O notation.

### Question 10

Determine the time complexity:

```
for(int i=1;i<=n;i++)
{}
for(int j=1;j<=n*n;j++)
{}
```

**Time Complexity: O(n²)**

**Explanation:** These are sequential loops. The first loop performs $n$ steps ($O(n)$). The second loop performs $n^2$ steps ($O(n^2)$). For sequential blocks, the overall asymptotic complexity is determined by the dominant, higher-order term: $O(n + n^2) = O(n^2)$.

### Question 11

Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n*n;j++)
    {
    }
}
```

**Time Complexity: O(n³)**

**Explanation:** These loops are nested. The outer loop runs $n$ times. For each outer iteration, the inner loop runs $n^2$ times. Multiplying the two independent counts together gives $n \times n^2 = n^3$ operations, or cubic complexity.

### Question 12

Determine the time complexity:

```
for(int i=1;i<=n;i*=3)
{}
```

**Time Complexity: O($\log_3 n$)**

**Explanation:** The control variable $i$ is scaled by a factor of 3 on every loop step ($i = 1, 3, 9, 27, ..., 3^k$). The loop stops when $3^k > n$, resulting in $k = \log_3 n$ steps. In Big-O, this can also be generalized as $O(\log n)$ since log bases differ only by a constant coefficient.

### Question 13

Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i*i;j++)
    {
    }
}
```

**Time Complexity: O($n^3$)**

**Explanation:** The outer loop runs from 1 to $n$. The inner loop executes $i^2$ times for a given $i$. The total number of steps is given by the sum of squares formula:

$$\Sigma_{i=1}^{n} i^2 = [n(n + 1)(2n + 1)] / 6 = (2n^3 + 3n^2 + n) / 6$$

The highest power of $n$ is $n^3$, so the asymptotic complexity is $O(n^3)$.

## Section 2: Recursive Code Analysis (Q14 - Q20)

### Question 14

Determine the time complexity:

```
void fun(int n)
{
    if(n==0)
        return;

    fun(n-1);
}
```

**Time Complexity: O(n)**

**Explanation:** The recurrence relation for this function is $T(n) = T(n-1) + O(1)$ with base case $T(0) = O(1)$. Unrolling or expanding this equation yields:

$$T(n) = T(n-2) + 1 + 1 = T(n-3) + 3 = \ldots = T(n-k) + k$$

For $k = n$, $T(n) = T(0) + n = 1 + n$, which matches $O(n)$.

### Question 15

Determine the time complexity:

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);
}
```

**Time Complexity: O(log n)**

**Explanation:** Each recursive step divides the input parameter $n$ by 2. The recurrence equation is $T(n) = T(n/2) + O(1)$. Using Master's Theorem ($a=1, b=2, f(n)=1$), we find $n^{\log_2 1} = n^0 = 1$, matching $f(n)$. Case 2 applies, giving $O(\log n)$.

### Question 16

Determine the time complexity:

```
void fun(int n)
{
    if(n==0)
        return;

    fun(n-1);
    fun(n-1);
}
```

**Time Complexity: $O(2^n)$**

**Explanation:** The function triggers two independent recursive calls of size $n-1$. This yields the recurrence relation $T(n) = 2T(n-1) + O(1)$. Expanding this expression shows a binary tree of depth $n$, where the total nodes double at each level: $1 + 2 + 4 + ... + 2^{n-1} = 2^n - 1$, resulting in $O(2^n)$ exponential time.

### Question 17

Determine the time complexity:

```
int fib(int n)
{
    if(n<=1)
        return n;

    return fib(n-1)+fib(n-2);
}
```

**Time Complexity: $O(2^n)$**

**Explanation:** This is the standard naive recursive implementation of the Fibonacci sequence. Its recurrence relation is $T(n) = T(n-1) + T(n-2) + O(1)$. The execution tree branches twice per step. The rigorous upper bound is bounded by the Golden Ratio $O(1.618^n)$, which is asymptotically categorized under loose exponential form as $O(2^n)$.

### Question 18

Determine the time complexity:

```
int fact(int n)
{
    if(n<=1)
        return 1;

    return n*fact(n-1);
}
```

**Time Complexity: O(n)**

**Explanation:** The recursive structure for calculating a factorial creates a linear sequence of calls: $T(n) = T(n-1) + O(1)$. It descends sequentially from $n$ down to 1, executing exactly $n$ nested activations, giving $O(n)$ linear time complexity.

### Question 19

Determine the time complexity:

```
void fun(int n)
{
    if(n<=0)
        return;

    fun(n-1);

    cout<<n;
}
```

**Time Complexity: O(n)**

**Explanation:** Printing the element $cout << n$ occurs after the recursive call (post-order tracking), taking constant $O(1)$ time. The sequence of execution flows lineally as $T(n) = T(n-1) + 1$, which yields $O(n)$.

### Question 20

Determine the time complexity:

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);

    cout<<n;
}
```

**Time Complexity: O(log n)**

**Explanation:** The presence of the print statement after the recursive call shifts printing sequence but does not change the operational count. The recurrence continues to follow $T(n) = T(n/2) + O(1)$, matching $O(log\ n)$ complexity.

## Section 3: Recurrence Relations (Q21 - Q25)

### Question 21

Solve the recurrence relation: $T(n) = T(n-1) + 1$

**Solution: T(n) = O(n)**

**Step-by-step Derivation via Substitution:**

- Let $T(n) = T(n-1) + 1$
- Substitute $T(n-1)$: $T(n) = [T(n-2) + 1] + 1 = T(n-2) + 2$
- Substitute $T(n-2)$: $T(n) = [T(n-3) + 1] + 2 = T(n-3) + 3$
- Generalizing after $k$ steps: $T(n) = T(n-k) + k$
- Assuming a base case of $T(0) = 1$, let $n - k = 0 ightarrow k = n$
- Result: $T(n) = T(0) + n = 1 + n = O(n)$

### Question 22

Solve the recurrence relation: $T(n) = T(n-1) + n$

**Solution: T(n) = O(n²)**

**Step-by-step Derivation:**

- Unrolling the relation: $T(n) = T(n-1) + n$
- $T(n) = [T(n-2) + (n-1)] + n = T(n-2) + n + (n-1)$
- $T(n) = T(n-3) + n + (n-1) + (n-2)$
- Continuing until base case $T(1)$: $T(n) = T(1) + \sum_{i=2}^{n} i$
- This is the sum of the first $n$ natural numbers: $\frac{n(n+1)}{2} = O(n^2)$

### Question 23

Solve the recurrence relation: $T(n) = T(n/2) + 1$

**Solution: T(n) = O(log n)**

**Step-by-step Derivation via Master Theorem:**

- Form: $T(n) = aT(n/b) + f(n)$, where $a = 1, b = 2, f(n) = 1$.
- Compute $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.
- Since $f(n) = 1 = \Theta(n^0)$, this falls precisely into **Case 2** of Master's Theorem.
- Formula: $T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(1 \cdot \log n) = O(\log n)$.

### Question 24

Solve the recurrence relation: $T(n) = 2T(n-1) + 1$

**Solution: T(n) = O(2ⁿ)**

**Step-by-step Derivation:**

- Expand the terms: $T(n) = 2[2T(n-2) + 1] + 1 = 4T(n-2) + 2 + 1$
- Next step: $T(n) = 4[2T(n-3) + 1] + 2 + 1 = 8T(n-3) + 4 + 2 + 1$

- General expression for $k$ steps: $T(n) = 2^k T(n-k) + \sum_{i=0}^{k-1} 2^i$
- Let $n - k = 0 \Rightarrow k = n$: $T(n) = 2^n T(0) + (2^n - 1) = O(2^n)$.

### Question 25

Solve the recurrence relation: $T(n) = T(n/2) + n$

**Solution: T(n) = O(n)**

**Step-by-step Derivation via Master Theorem:**

- Parameters: $a = 1, b = 2, f(n) = n$.
- Compute threshold: $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.
- Compare $f(n) = n^1$ with $n^0$. Since $f(n) = \Omega(n^{0 + \epsilon})$ for $\epsilon = 1$, this falls into **Case 3**.
- Check regularity condition: $a \cdot f(n/b) \le c \cdot f(n) \Rightarrow 1 \cdot (n/2) \le c \cdot n$. True for $c = 0.5$.
- Therefore, $T(n) = \Theta(f(n)) = O(n)$.

## Section 4: Bonus GATE-Level Questions (Q26 - Q30)

### Question 26

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i;j*=2)
    {
    }
}
```

**Time Complexity: O(n)**

**Explanation:** The outer loop progresses linearly from 1 to $n$. The inner loop doubles $j$ until it exceeds $i$, thus executing $\log_2(i)$ times for each step $i$. The total operations sum up to:

$$\sum_{i=1}^{n} \log(i) = \log(1) + \log(2) + ... + \log(n) = \log(n!)$$

By Stirling's Approximation, $\log(n!) \approx n \log n - n$. Thus, the exact tight asymptotic bound is $O(n \log n)$.

### Question 27

```
for(int i=1;i<=n;i*=2)
{
    for(int j=1;j<=n;j++)
    {
    }
}
```

**Time Complexity: O(n log n)**

**Explanation:** The outer loop runs in logarithmic scale because $i$ doubles each step, executing $log_2 n$ times. The inner loop is independent of $i$ and always executes exactly $n$ times. Multiplying the two execution limits gives $O(n \cdot log n)$.

### Question 28

```
for(int i=n;i>=1;i--)
{
    for(int j=i;j>=1;j--)
    {
    }
}
```

**Time Complexity: O(n²)**

**Explanation:** This loop works in reverse but is functionally matching Q3. The outer loop counts down from $n$ to 1. The inner loop executes $i$ times for each corresponding $i$. The total operation count matches the sum sequence: $n + (n-1) + (n-2) + ... + 1 = rac{n(n+1)}{2} = O(n^2)$.

### Question 29

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j*=2)
    {
    }
}
```

**Time Complexity: O(n log n)**

**Explanation:** The outer loop is linear and iterates exactly $n$ times. The inner loop scales exponentially with $j*=2$ up to a ceiling of $n$, executing $log_2\ n$ times. Because the two loops are independent, the compound runtime is the product: $O(n\ log\ n)$.

### Question 30

```
void fun(int n)
{
    if(n<=1)
        return;

    fun(n/2);
    fun(n/2);
}
```

**Time Complexity: O(n)**

**Explanation:** This function forks into two recursive paths, each taking half the input scale. The representative recurrence equation is $T(n) = 2T(n/2) + O(1)$. Applying Master's Theorem: $a = 2, b = 2, f(n) = 1$. Compute $n^{log_b a} = n^{log_2 2} = n^1 = n$. Since $f(n) = 1 = O(n^{1 - \epsilon})$ for $\epsilon = 1$, it falls under **Case 1** of the theorem. Therefore, $T(n) = \Theta(n^{log_b a}) = O(n)$.